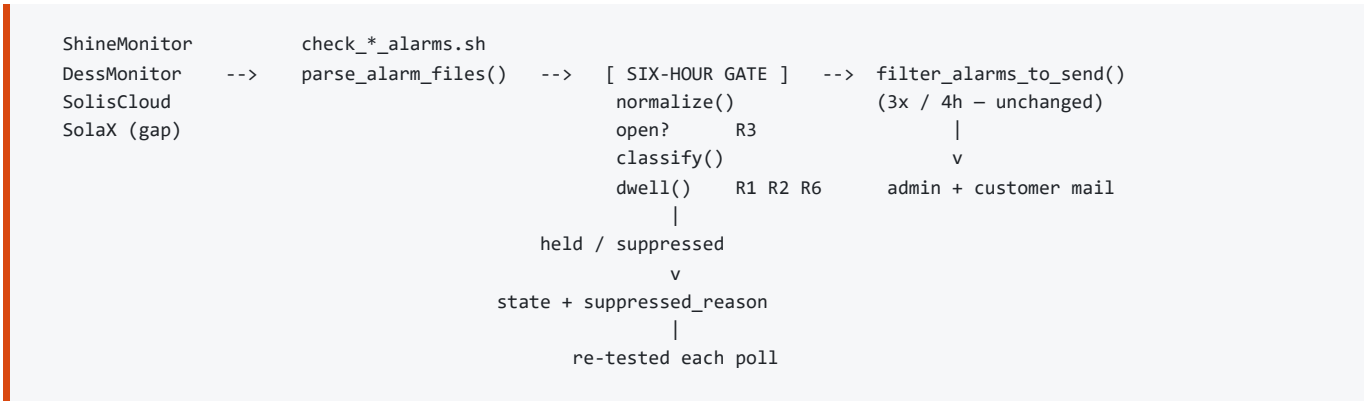
Six-Hour Gate — alarm triage for the IoT platform

Decides which device alarms are worth a human's attention. One rule: an alarm has to **still be wrong six hours later**. Everything the fleet shouts about in between is recorded in state and never mailed.

Escalations go to the admin (`ktronicssolar@gmail.com`), not to customers. The gate does not depend on customer email addresses — only 6 of 23 ShineMonitor accounts and 1 of 3 DessMonitor accounts have one, and that is irrelevant to whether an alarm escalates. Customer-facing device-alarm mail is a separate, independently flagged channel (`emails.device_alarms_customer`).



The gate is a **pre-filter, not a rewrite**. The three-sends-then-ignore behaviour stays exactly as it is — an alarm that never qualifies simply never spends one of its three sends.

Status

Design	Complete — full design doc
Code	Live. Wired into `generate_device_alarms.py` behind the `alerts.six_hour_gate` flag, `on` since 11 Oct 2026
Config	`config/alarm_triage.json` — class map, dwell hours, matchers
Blocking	Nothing. R3 (status filter) shipped with the gate
Uncovered	SolaX — web login only, no API key. Solis — adapter not written yet

Why

Read from `state/device_alarms_state.json` + `state/dessmonitor_device_alarms_state.json` (9 Sep 2026), across 25 accounts and 21 plants:

142	alarms tracked
105	of them battery-voltage — 57 "too low", 26 "Low battery", 18 "too high", 4 charger-stops
121	already mailed 3× and permanently silenced, fixed or not

A battery crossing a voltage threshold is what a battery on an off-grid or grid-scarce system does **every night**. The five records saying "*Battery connection is open*" — a genuinely dead pack — arrived in the same inbox, same format, same priority as the 57 saying the battery got low after dark.

SolisCloud says the same from the other side: 44 plants, 33 online, 4 in alarm — three of those four are `1015 NO-Grid`, a CEB outage that clears itself before anyone reads the mail.

The rules

	Rule	Why it matters
R1	Dwell is `now − alarm_start` from the **vendor's** timestamp, not our `first_seen`. Absence from the feed = cleared	An alarm already 12 h old at first poll counts as 12 h old. `first_seen` is fallback only
R2	Convert `gts` from **plant-local** (UTC+5:30) before comparing to a UTC runner	A naive `datetime.now() - gts` under-reports age by exactly 5.5 h — the "6-hour" rule would silently behave as **11.5 h**, invisibly. Use `queryPlants -> address.timezone`, same as `check_ph1000.py:486`
R3	Count only alarms that are **actually open** — `status` false/0, Solis `state == 1`	**Shipped first.** The fetch scripts used to run in "TESTING MODE" (`date=`, no status filter), so *resolved* alarms came back. A dwell gate on that feed would have turned every stale fixed alarm into a fresh call-out. They now ask for `status=0`, and anything not open is dropped again at parse time
~~**R4**~~	~~Offline-plant suppression + weekly offline digest~~ — **withdrawn 13 Sep 2026**	Offline is already detected downstream: a dark logger writes `0.0000` rows, so a sustained outage trips the **3-day production-loss RED alert** on the CSV side. The gate needs no digest and no offline-plant rule of its own. `COMMS` *alarms are still never mailed* — that lives in the class map, not as a rule. Number kept vacant so R5/R6 references stay stable
~~**R5**~~	~~`SOC` alarms are measured in daylight hours — must survive 09:00–15:00 local still open~~ — **withdrawn 12 Oct 2026**	Battery alarms are on the same clock as everything else (R6): one rule to explain, one to test. **The cost is known:** "Battery too low" at 22:00 that clears at 07:00 is 9 h old, so it now mails at about 04:00, on every night a plant does that. Number kept vacant, as with R4
R6	**One clock for every class** — an alarm that outlives 6 h triggers an email; under 6 h it is silent. Grid loss and battery alarms get no special case	A 2–4 h CEB cut never reaches an inbox. What outlives 6 h is usually a tripped AC input breaker or blown input fuse — our call-out, not the utility's

If battery mail gets noisy: give `classes.SOC` a longer `dwell_hours` — 14 outlasts a 22:00→07:00 overnight dip. One config line, no code, at the cost of hearing about a genuinely bad December night eight hours later.

Feature flags

Every outbound email is gated by `config/features.json`, read by `features.py` (Python), `feature_enabled()` in `common_config.sh` (bash), and a `Load feature flags` step in each workflow. Turning a channel off needs a one-line config change, not a code deletion.

Gating happens in two places. Workflow steps that shell out to `send_email.py` carry an `if:` on `steps.flags.outputs.*`. Scripts that mail from inside Python are gated at the chokepoint instead: **every** outbound email in this repo goes through `email_utils.send_email_smtp()`, which takes a `channel=` naming the flag that gates it (or `WORKFLOW_GATED` when the workflow step already does). Two BVT tests keep that true — one fails if any script imports `smtplib` directly, one fails if any `send_email_smtp()` call omits `channel=`.

Current profile — critical only (set 13 Sep 2026; the connectivity and gate flags were added since):

Channel	State
`emails.weekly_customer_reports`	on
`emails.weekly_admin_summary`	**off**
`emails.device_alarms_admin` / `_customer`	on
`emails.connectivity_admin`	on
`emails.production_alerts_admin` / `_customer`	on
`emails.build_verification`, `emails.test_regression`	on
`emails.solis_switch_admin`	on (workflow dormant)
`emails.bms_alerts_admin`	on (no hardware deployed)
`alerts.production_red_3day`	on
`alerts.production_orange_3month`	**off**
`alerts.connectivity_link_down`	on
`alerts.six_hour_gate`	on (three states: `off`, `dry_run`, `on`)

Override for a single run without a commit — the env var is `KT_FEATURE_` plus the dotted path, upper-cased:

```
# one-off: send this week's customer reports even though the flag is off
KT_FEATURE_EMAILS_WEEKLY_CUSTOMER_REPORTS=true \
  python3 src/main/java/org/ktronics/scripts/send_customer_emails.py --reports-dir reports

python3 src/main/java/org/ktronics/scripts/features.py     # print the live flag state
```

Both weekly workflows short-circuit at **job** level: with both weekly flags off they no-op instead of generating reports nobody receives, so no CI minutes are spent.

Everything **fails open** — a missing or corrupt `features.json`, or an unknown flag name, reads as *on*. A broken config must never silence alerting. Build-failure mail additionally uses `!= 'false'` rather than `== 'true'`, so a crash before the flags step still notifies. The gate is the one flag that *suppresses*, so it reads the other way: missing or unparseable means `off`, and nothing is held.

What this profile costs — reviewed 13 Sep 2026

Every flag here was checked against the use cases it overrides. Three were resolved, one was accepted with eyes open, one remains.

	Concern	Outcome
✅	Offline plants had no reporting channel with the digest off	**Resolved by removing R4.** A dark logger writes `0.0000` rows, so a sustained outage trips the 3-day production-loss alert. One alert path, not two
✅	UC2 (Customer Weekly Reports) was dark	**Re-enabled.** Customers get their Sunday summary again

	Concern	Outcome
✅	UC8 was dead logic; UC4 lost its canary	**Both restored by re-enabling UC2.** UC8's `event_name == 'schedule'` condition is live again, and because the job no longer short-circuits, report generation runs on every push — so a regression in `generate_weekly_report.py` still fails the build immediately
⚠️	ORANGE (3-month) is off	**Accepted.** See below
⚠️	`weekly_admin_summary` is off	No admin-side weekly fleet view. Not raised as a concern; flip the flag if you want it back

The accepted cost — ORANGE. RED only fires below 20% of baseline for 3 consecutive days: a plant that has essentially stopped. A plant sitting at 45% for three months — soiling, a dead string, a derating inverter, a tree that grew — never trips RED. ORANGE was the only rule catching quiet revenue loss, and per year it is plausibly worth more than RED. It is off by decision, not by accident. `--orange` forces it on for a manual run, and the UC1 tests pin it so the logic stays covered and green for whenever it comes back.

Class map

Six classes, two gates. Counts are the state files of 9 Sep 2026. `COMMS` is the "ignore offline" rule from the original brief: never mailed, never digested — if a logger stays dark it shows up as a 3-day production-loss alert instead.

Class	How our fleet words it	Gate	Today
`COMMS`	offline · datalogger lost · device disconnected	**never mail**	—
`SOC`	battery voltage too low (57) · Low battery (26) · too high (18) · charger stops, low battery (4)	**6 h clock**	105
`HARDWARE`	bus voltage too low (7) · battery connection open (5) · bus soft start failed (3) · sensor error · voltage class error	**6 h clock**	17
`LOAD`	over current protection (9) · overload time out (2) · charger stops, over load (2) · overload	**6 h clock**	14
`THERMAL`	radiator over temperature · fan locked (on) · fan locked (off)	**6 h clock**	3
`GRID`	`1015 NO-Grid` (Solis) · grid under frequency · utility loss	**6 h clock**	1
`PV`	charger stops, high PV voltage	**6 h clock**	1

Unrecognised messages fall to `UNKNOWN` and take the 6-hour clock — the safe default is to let a novel fault through, then add it to the map. Every `UNKNOWN` is logged so the map grows from real traffic, not guesswork.

Vendor coverage

Each vendor gets a thin adapter producing the same record — `{plant, device, vendor_code, message, class, started_at_utc, open, customer}` — and the gate never learns a vendor's name.

Inverter	Portal	Alarm source	Start time	Open flag	State
Voltronic / PowMr	ShineMonitor	`webQueryPlantsWarning`	`gts` · plant-local	`status = false`	live behind the gate

Inverter	Portal	Alarm source	Start time	Open flag	State
Deye	DessMonitor	same shape, `dessmonitor-*`	`gts` · plant-local	`status = false`	live behind the gate
Solis	SolisCloud	`POST /v1/api/alarmList`	`alarmBeginTime` · epoch ms	`state = 1`	adapter to write — needs an API key first
SolaX	SolaX Cloud	— portal login only	—	—	**not covered**

Solis is cheap: `solis_common.py` already signs and posts arbitrary resources, so `alarmList` is a method plus a field map. Ktronics AB is the distributor on that account, but no API key has been issued yet, so nothing Solis has run.

SolaX is the weak link — say so out loud. We have a web login and no API key, so SolaX plants are currently unmonitored by anything automated. Note that even the public SolaX Cloud token API returns `getRealtimeInfo` with an `inverterStatus` code and live values — **not** an alarm list with start times. A token alone would not satisfy R1; we'd have to derive alarms from `inverterStatus` transitions and time the dwell from our own `first_seen`, which is the weaker fallback R1 allows. Until then SolaX stays on manual portal review. Note there is no digest to flag them in — SolaX plants are not in `data/` either, so the 3-day production-loss path does not cover them the way it covers offline loggers elsewhere. **SolaX is genuinely unmonitored, and silence from it means nothing.**

What changed

File	Change
`six-hour-gate/alarm_triage.py`	`normalize()`, `classify()`, `is_actionable()`. Pure functions, no I/O
`six-hour-gate/config/alarm_triage.json`	dwell hours per class, COMMS list
`generate_device_alarms.py`	`triage_alarms()` runs between `parse_alarm_files()` and `filter_alarms_to_send()`. Rest untouched
`check_device_alarms.sh` `check_dessmonitor_device_alarms.sh`	TESTING MODE dropped, feed filtered to unhandled (R3), raw-response `DEBUG` dump removed. The scripts still print the response body when a fetch fails — not done yet
`state/*_device_alarms_state.json`	`class`, `started_at_utc`, `suppressed_reason` — so "*why wasn't I told about this?*" is answerable from state alone. Written only for alarms the gate holds — not done yet for the rest
`trigger-shinemonitor.yml` `trigger-dessmonitor.yml`	alarm poll is hourly; the CSV data pull stays at 4 h

Tests

The gate is additive: the UC3 tests pass untouched. Its own tests are named for the rule they defend — `test_alarm_triage.py` for the rules, `test_alarm_gate_wiring.py` for where the gate plugs in:

- **R1** — escalates at 6 h 01 m, silent at 5 h 59 m. *Not tested yet:* an alarm absent from the feed clears
- **R2** — plant-local `gts` against a UTC runner yields the true age *(the regression that would otherwise hide for months)*
- **R3** — a handled alarm never escalates, however old
- **COMMS** — an offline/comms alarm is never mailed, at any age
- **R6** — a 3-hour grid cut is silent; a 7-hour one escalates. A battery alarm is held to the same clock, day or night: silent at 5 h 59 m, mailed at 6 h 01 m

The CSV side

Six hours doesn't map to a once-a-day CSV, but the same question does. `check_anomaly.py` already has a persistence bar (RED = under 20% of baseline for 3 consecutive days); what's missing is *is this actually ours?*

1. **Offline detection rides on the 3-day rule — deliberately.** A dark datalogger writes `0.0000` rows, which *is* the RED condition, so a logger that stays down for three days raises a production alert by itself. **This is the design, not a bug** (decision of 13 Sep 2026, replacing R4): it means no separate offline monitor, no digest, and one alert path instead of two.

   The cost is that the CSV alone cannot tell *no data* from *no production* — a dead router and a dead inverter look identical. So the RED escalation text says so: "*no data or no production for 3 days — check the datalogger is online before dispatching.*" Without that line a technician drives out to a plant that was working fine. `--ignore-zero-months` still ignores plants dark for a full month, which is the right long-tail behaviour.

2. **Gate on the fleet, not the plant's own baseline.** A dull week drags every plant down together and should mail nobody. `fleet_benchmark.py` already builds an irradiance proxy from the customer's own neighbours — wire it in so a RED alert mails only if the plant underperformed *the fleet*.

3. **Don't mail the same fault twice.** If a plant already has an escalated device alarm explaining the shortfall, the production alert becomes a line inside that email.

Order of work

The first item was not optional and not reorderable. Steps 1, 2 and 4 are done.

1. **R3 — the status filter.** *Done.* Stop pulling resolved alarms. Had to land before or with the gate
2. **R2 + R1 — the gate itself.** *Done, live since 11 Oct 2026.* Timezone conversion, six-hour dwell, and the `COMMS` never-mail class. The core commit
3. ~~**R5 — the daylight clock for SOC.**~~ Withdrawn 12 Oct 2026 — battery alarms ride the six-hour clock from step 2
4. **Hourly alarm poll.** *Done.* Makes the six hours literal instead of six-to-ten
5. **Solis adapter.** *Waiting on an API key.* `alarmList` against the existing signed client — brings 44 plants under the same rule
6. **CSV gating.** *Half done.* The "check the logger is online" line is in the RED escalation text; the fleet-benchmark gate is not built
7. **SolaX API token.** Request in parallel — a procurement wait, not engineering work
8. **SolaX adapter.** After the token, and only if the decision is to derive alarms from `inverterStatus` instead of keeping SolaX on manual portal review

Expected effect

Before — 142 alarms, each mailed three times regardless of merit; 121 already permanently silenced whether or not the fault was ever fixed.

After — every alarm, battery included, mails only if it is still open six hours on, with a plant name and a first thing to check. Short dips never reach an inbox; an overnight battery dip that lasts until sunrise still does (the accepted cost of withdrawing R5). Comms faults never mail. On Solis, four open alarms become three `NO-Grid` on a clock most CEB cuts never reach, plus one battery alarm on the same clock.

The number that matters is not how many emails we send. It's that when one arrives, someone gets in the van.